

第一、二章

名词解释

软件工程

简单地理解，软件工程就是生产软件的工程学。

现在应用较为广泛的软件工程定义为：

- 1) 应用系统的、规范的、可量化的方法来开发、运行和维护软件，即将工程应用到软件。
- 2) 对 1) 中各种方法的研究

事实上，软件工程是一个包含复杂内容的计算机学子学科，其特性不是通过一个或几个有限定义所能概括的。

简答

从 1950s~2000s 之间的特点

1950s: 科学计算；以机器为中心进行编程；像生产硬件一样生产软件。

1960s: 业务应用（批量数据处理和事务计算）；软件不同于硬件；用软件工艺的方式生产软件。

1970s: 结构化方法；瀑布模型；强调规则和纪律。

1980s: 追求生产力最大化；现代结构化方法/面向对象编程广泛应用；重视过程的作用。

1990s: 企业为中心的大规模软件系统开发；追求快速开发、可变更性和用户价值；Web 应用出现。

2000s: 大规模 Web 应用；大量面向大众的 Web 产品，追求快速开发、可变更性、用户价值和 innovation。

第四章

如何管理团队？

在实验中采取了哪些方法？有哪些经验？

团队结构有哪几种？

1. 主程序员团队
2. 民主团队
3. 开放团队

质量保障有哪些措施？结合实验进行说明

1. 需求开发：需求评审、需求度量
2. 体系结构：体系结构评审、集成测试（持续集成）
3. 详细设计：详细设计评审、设计度量、集成测试（持续集成）
4. 实现（构造）：代码评审、代码度量、测试（测试驱动、持续集成）
5. 测试：测试、测试度量

配置管理有哪些活动？实验中是如何进行配置管理的？ P60

- (1) 标识配置项
- (2) 版本管理
- (3) 变更控制
- (4) 配置审计

- (5) 状态报告
- (6) 软件发布管理

第五章

名词解释

需求

- 1)用户为了解决问题或者达到某些目标所需要的条件或能力
- 2)系统或系统部件为了满足合同、标准、规范或其他正式文档所规定的要求而需要具备的条件或能力
- 3)对 1)或 2)中的一个条件或一种能力的一种文档化表述

需求工程就是所有需求处理活动的总和，它收集信息、分析问题、整合观点、记录需求并验证其正确性，最终描述出软件被应用后与其环境互动形成的期望效应。

区分需求的三个层次 P72

给出一个实例给出其三个层次的例子

对给定的需求示例，判定其层次，例如课程实验/ATM/图书管理...

1.业务需求

是系统建立的战略出发点，表现为高层次的目标，它描述了组织为什么要开发系统。

2.用户需求

是执行实际工作的用户对系统所能完成的具体任务的期望，描述了系统能够帮助用户做些什么。

3.系统级需求

是用户对系统行为的期望，每个系统级需求反映了一次外界与系统的交互行为，或者系统的一个实现细节。

掌握需求的类型

对给定的实例，给出其不同类型的需求例子

对给定的需求示例，判断其类型，例如课程实验/ATM/图书管理...

1.功能需求

2.性能需求

常见：速度，容量，吞吐量，负载，实时性

3.质量属性

可靠性，可用性，安全性，可维护性，可移植性，易用性

4.对外接口

用户界面、硬件接口、软件接口、网络通信接口等

5.约束

进行系统构造时需要遵守的约定，如编程语言、硬件设施等，常见：系统开发及其运行的环境，问题域内的相关标准（包括法律法规、行业协定、企业规章等），商业规则

6.数据需求

各个功能使用的数据信息，使用频率，可访问性要求，数据实体及其关系，完整性约束，数据保持要求

第六章

为给定的描述

建立用例图

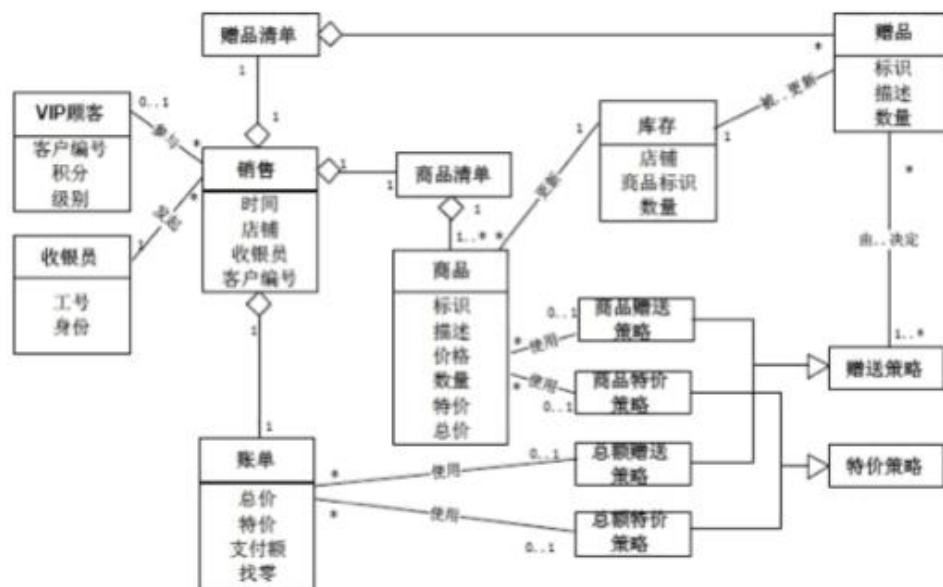
- 1)进行目标分析与确定解决方向

- 2)寻找参与者
- 3)寻找用例
- 4)细化用例
- 5)注意事项：不要将用例细化为单个操作，不要将同一个业务目标细化为不同用例，不要将没用业务价值的内容作为用例

建立分析类图

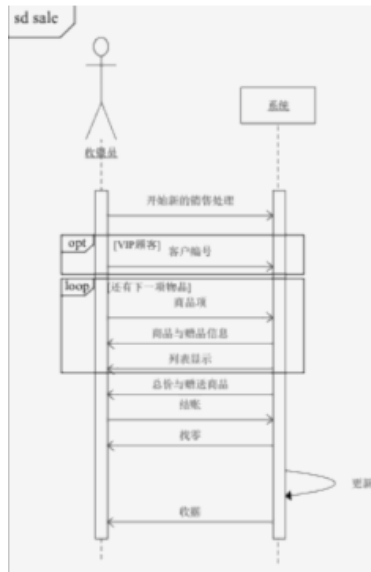
概念类图又称为领域模型，关注系统与外界的交互

- 1)识别候选类
- 2)确定概念类
- 3)识别关联
- 4)识别重要属性



建立系统顺序图

- 1)确定上下文环境
- 2)发现交互对象
- 3)根据用例描述中的流程，逐步添加信息



如果一个对象发送了一个同步消息，那么它要等待对方对消息的应答，收到应答后才能继续自己的操作。而发送异步消息的对象不需要等待对方对消息的应答便可以继续自己的操作。

建立状态图

- 1)确定上下文环境，搞清楚状态的主体
- 2)识别状态
- 3)建立状态转换
- 4)补充详细信息

- 为下列描述建立用例模型（6分），要求明确给出建模过程（4分）。
- 现在需要开发一个简化了的大学图书馆系统，它有几种类型的借书人，包括教职工借书人、研究生借书人和本科生借书人等。借书人的基本信息包括姓名、地址和电话号码等。对于教职工借书人，还要包括诸如办公室地址和电话等信息。对于研究生借书人，还要包括研究项目和导师信息等。对于本科生借书人，还要包括项目和所有学分信息等。
- 图书馆系统要跟踪借出书本信息。当一个借书人捧着一堆书去借书台办理借书手续时，借出这个事件就发生了。随着时间的过去，一个借书人可以多次从图书馆中借书。一次可以借出多本图书。
- 如果借书人想要的书已被借出，他可以预约。每个预约只针对一个借书人和一个标题。预约日期、优先权和完成日期等信息需要维护。当借书完成，系统会将这本书与借出联系起来。
- 借书人根据图书馆的信息来检索书名，同时检索这本书是否可以被借出。如果一本书的所有副本都被借出了，那么借书人可以根据书名预订这本书。当借书人把书拿到借书台的时候，管理员可以为这些书办理归还手续。管理员要跟踪新书到达的情况。
- 图书馆的管理者有属于自己的活动。他们要分类打出关于书的标题的表格，还要在线检查所有过期未还的书，也标出来。而且，图书馆系统还可以从另外一个大学的数据库中访问和下载借书人的信息。

- 下面是一段需求描述，请依据其建立ATM机系统的领域类图（即分析类图）。
- A银行计划在B大学开设银行分部，计划使用ATM机提供全部服务。ATM系统将通过显示屏幕、输入键盘（有数字键和特殊符号键）、银行卡读卡器、存款插槽、收据打印机等设备与客户交互。客户可以使用ATM机进行存款、取款、余额查询等操作，它们对帐户的更新将交由账户系统的一个接口来处理。安全系统将为每个客户分配一个PIN码和安全级别。每次事务执行之前都需要验证该PIN码。在将来，银行还计划使用ATM机支持一些常规的操作，例如地址和电话号码修改。

在一栋大楼中有一个电梯，现在有人打算从1楼使用电梯到达x楼。

- 请写出其用例的场景描述
- 根据用例场景编写SRS片段

... 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100. 101. 102. 103. 104. 105. 106. 107. 108. 109. 110. 111. 112. 113. 114. 115. 116. 117. 118. 119. 120. 121. 122. 123. 124. 125. 126. 127. 128. 129. 130. 131. 132. 133. 134. 135. 136. 137. 138. 139. 140. 141. 142. 143. 144. 145. 146. 147. 148. 149. 150. 151. 152. 153. 154. 155. 156. 157. 158. 159. 160. 161. 162. 163. 164. 165. 166. 167. 168. 169. 170. 171. 172. 173. 174. 175. 176. 177. 178. 179. 180. 181. 182. 183. 184. 185. 186. 187. 188. 189. 190. 191. 192. 193. 194. 195. 196. 197. 198. 199. 200. 201. 202. 203. 204. 205. 206. 207. 208. 209. 210. 211. 212. 213. 214. 215. 216. 217. 218. 219. 220. 221. 222. 223. 224. 225. 226. 227. 228. 229. 230. 231. 232. 233. 234. 235. 236. 237. 238. 239. 240. 241. 242. 243. 244. 245. 246. 247. 248. 249. 250. 251. 252. 253. 254. 255. 256. 257. 258. 259. 260. 261. 262. 263. 264. 265. 266. 267. 268. 269. 270. 271. 272. 273. 274. 275. 276. 277. 278. 279. 280. 281. 282. 283. 284. 285. 286. 287. 288. 289. 290. 291. 292. 293. 294. 295. 296. 297. 298. 299. 300. 301. 302. 303. 304. 305. 306. 307. 308. 309. 310. 311. 312. 313. 314. 315. 316. 317. 318. 319. 320. 321. 322. 323. 324. 325. 326. 327. 328. 329. 330. 331. 332. 333. 334. 335. 336. 337. 338. 339. 340. 341. 342. 343. 344. 345. 346. 347. 348. 349. 350. 351. 352. 353. 354. 355. 356. 357. 358. 359. 360. 361. 362. 363. 364. 365. 366. 367. 368. 369. 370. 371. 372. 373. 374. 375. 376. 377. 378. 379. 380. 381. 382. 383. 384. 385. 386. 387. 388. 389. 390. 391. 392. 393. 394. 395. 396. 397. 398. 399. 400. 401. 402. 403. 404. 405. 406. 407. 408. 409. 410. 411. 412. 413. 414. 415. 416. 417. 418. 419. 420. 421. 422. 423. 424. 425. 426. 427. 428. 429. 430. 431. 432. 433. 434. 435. 436. 437. 438. 439. 440. 441. 442. 443. 444. 445. 446. 447. 448. 449. 450. 451. 452. 453. 454. 455. 456. 457. 458. 459. 460. 461. 462. 463. 464. 465. 466. 467. 468. 469. 470. 471. 472. 473. 474. 475. 476. 477. 478. 479. 480. 481. 482. 483. 484. 485. 486. 487. 488. 489. 490. 491. 492. 493. 494. 495. 496. 497. 498. 499. 500. 501. 502. 503. 504. 505. 506. 507. 508. 509. 510. 511. 512. 513. 514. 515. 516. 517. 518. 519. 520. 521. 522. 523. 524. 525. 526. 527. 528. 529. 530. 531. 532. 533. 534. 535. 536. 537. 538. 539. 540. 541. 542. 543. 544. 545. 546. 547. 548. 549. 550. 551. 552. 553. 554. 555. 556. 557. 558. 559. 560. 561. 562. 563. 564. 565. 566. 567. 568. 569. 570. 571. 572. 573. 574. 575. 576. 577. 578. 579. 580. 581. 582. 583. 584. 585. 586. 587. 588. 589. 590. 591. 592. 593. 594. 595. 596. 597. 598. 599. 600. 601. 602. 603. 604. 605. 606. 607. 608. 609. 610. 611. 612. 613. 614. 615. 616. 617. 618. 619. 620. 621. 622. 623. 624. 625. 626. 627. 628. 629. 630. 631. 632. 633. 634. 635. 636. 637. 638. 639. 640. 641. 642. 643. 644. 645. 646. 647. 648. 649. 650. 651. 652. 653. 654. 655. 656. 657. 658. 659. 660. 661. 662. 663. 664. 665. 666. 667. 668. 669. 670. 671. 672. 673. 674. 675. 676. 677. 678. 679. 680. 681. 682. 683. 684. 685. 686. 687. 688. 689. 690. 691. 692. 693. 694. 695. 696. 697. 698. 699. 700. 701. 702. 703. 704. 705. 706. 707. 708. 709. 710. 711. 712. 713. 714. 715. 716. 717. 718. 719. 720. 721. 722. 723. 724. 725. 726. 727. 728. 729. 730. 731. 732. 733. 734. 735. 736. 737. 738. 739. 740. 741. 742. 743. 744. 745. 746. 747. 748. 749. 750. 751. 752. 753. 754. 755. 756. 757. 758. 759. 760. 761. 762. 763. 764. 765. 766. 767. 768. 769. 770. 771. 772. 773. 774. 775. 776. 777. 778. 779. 780. 781. 782. 783. 784. 785. 786. 787. 788. 789. 790. 791. 792. 793. 794. 795. 796. 797. 798. 799. 800. 801. 802. 803. 804. 805. 806. 807. 808. 809. 810. 811. 812. 813. 814. 815. 816. 817. 818. 819. 820. 821. 822. 823. 824. 825. 826. 827. 828. 829. 830. 831. 832. 833. 834. 835. 836. 837. 838. 839. 840. 841. 842. 843. 844. 845. 846. 847. 848. 849. 850. 851. 852. 853. 854. 855. 856. 857. 858. 859. 860. 861. 862. 863. 864. 865. 866. 867. 868. 869. 870. 871. 872. 873. 874. 875. 876. 877. 878. 879. 880. 881. 882. 883. 884. 885. 886. 887. 888. 889. 890. 891. 892. 893. 894. 895. 896. 897. 898. 899. 900. 901. 902. 903. 904. 905. 906. 907. 908. 909. 910. 911. 912. 913. 914. 915. 916. 917. 918. 919. 920. 921. 922. 923. 924. 925. 926. 927. 928. 929. 930. 931. 932. 933. 934. 935. 936. 937. 938. 939. 940. 941. 942. 943. 944. 945. 946. 947. 948. 949. 950. 951. 952. 953. 954. 955. 956. 957. 958. 959. 960. 961. 962. 963. 964. 965. 966. 967. 968. 969. 970. 971. 972. 973. 974. 975. 976. 977. 978. 979. 980. 981. 982. 983. 984. 985. 986. 987. 988. 989. 990. 991. 992. 993. 994. 995. 996. 997. 998. 999. 1000. 1001. 1002. 1003. 1004. 1005. 1006. 1007. 1008. 1009. 1010. 1011. 1012. 1013. 1014. 1015. 1016. 1017. 1018. 1019. 1020. 1021. 1022. 1023. 1024. 1025. 1026. 1027. 1028. 1029. 1030. 1031. 1032. 1033. 1034. 1035. 1036. 1037. 1038. 1039. 1040. 1041. 1042. 1043. 1044. 1045. 1046. 1047. 1048. 1049. 1050. 1051. 1052. 1053. 1054. 1055. 1056. 1057. 1058. 1059. 1060. 1061. 1062. 1063. 1064. 1065. 1066. 1067. 1068. 1069. 1070. 1071. 1072. 1073. 1074. 1075. 1076. 1077. 1078. 1079. 1080. 1081. 1082. 1083. 1084. 1085. 1086. 1087. 1088. 1089. 1090. 1091. 1092. 1093. 1094. 1095. 1096. 1097. 1098. 1099. 1100. 1101. 1102. 1103. 1104. 1105. 1106. 1107. 1108. 1109. 1110. 1111. 1112. 1113. 1114. 1115. 1116. 1117. 1118. 1119. 1120. 1121. 1122. 1123. 1124. 1125. 1126. 1127. 1128. 1129. 1130. 1131. 1132. 1133. 1134. 1135. 1136. 1137. 1138. 1139. 1140. 1141. 1142. 1143. 1144. 1145. 1146. 1147. 1148. 1149. 1150. 1151. 1152. 1153. 1154. 1155. 1156. 1157. 1158. 1159. 1160. 1161. 1162. 1163. 1164. 1165. 1166. 1167. 1168. 1169. 1170. 1171. 1172. 1173. 1174. 1175. 1176. 1177. 1178. 1179. 1180. 1181. 1182. 1183. 1184. 1185. 1186. 1187. 1188. 1189. 1190. 1191. 1192. 1193. 1194. 1195. 1196. 1197. 1198. 1199. 1200. 1201. 1202. 1203. 1204. 1205. 1206. 1207. 1208. 1209. 1210. 1211. 1212. 1213. 1214. 1215. 1216. 1217. 1218. 1219. 1220. 1221. 1222. 1223. 1224. 1225. 1226. 1227. 1228. 1229. 1230. 1231. 1232. 1233. 1234. 1235. 1236. 1237. 1238. 1239. 1240. 1241. 1242. 1243. 1244. 1245. 1246. 1247. 1248. 1249. 1250. 1251. 1252. 1253. 1254. 1255. 1256. 1257. 1258. 1259. 1260. 1261. 1262. 1263. 1264. 1265. 1266. 1267. 1268. 1269. 1270. 1271. 1272. 1273. 1274. 1275. 1276. 1277. 1278. 1279. 1280. 1281. 1282. 1283. 1284. 1285. 1286. 1287. 1288. 1289. 1290. 1291. 1292. 1293. 1294. 1295. 1296. 1297. 1298. 1299. 1300. 1301. 1302. 1303. 1304. 1305. 1306. 1307. 1308. 1309. 1310. 1311. 1312. 1313. 1314. 1315. 1316. 1317. 1318. 1319. 1320. 1321. 1322. 1323. 1324. 1325. 1326. 1327. 1328. 1329. 1330. 1331. 1332. 1333. 1334. 1335. 1336. 1337. 1338. 1339. 1340. 1341. 1342. 1343. 1344. 1345. 1346. 1347. 1348. 1349. 1350. 1351. 1352. 1353. 1354. 1355. 1356. 1357. 1358. 1359. 1360. 1361. 1362. 1363. 1364. 1365. 1366. 1367. 1368. 1369. 1370. 1371. 1372. 1373. 1374. 1375. 1376. 1377. 1378. 1379. 1380. 1381. 1382. 1383. 1384. 1385. 1386. 1387. 1388. 1389. 1390. 1391. 1392. 1393. 1394. 1395. 1396. 1397. 1398. 1399. 1400. 1401. 1402. 1403. 1404. 1405. 1406. 1407. 1408. 1409. 1410. 1411. 1412. 1413. 1414. 1415. 1416. 1417. 1418. 1419. 1420. 1421. 1422. 1423. 1424. 1425. 1426. 1427. 1428. 1429. 1430. 1431. 1432. 1433. 1434. 1435. 1436. 1437. 1438. 1439. 1440. 1441. 1442. 1443. 1444. 1445. 1446. 1447. 1448. 1449. 1450. 1451. 1452. 1453. 1454. 1455. 1456. 1457. 1458. 1459. 1460. 1461. 1462. 1463. 1464. 1465. 1466. 1467. 1468. 1469. 1470. 1471. 1472. 1473. 1474. 1475. 1476. 1477. 1478. 1479. 1480. 1481. 1482. 1483. 1484. 1485. 1486. 1487. 1488. 1489. 1490. 1491. 1492. 1493. 1494. 1495. 1496. 1497. 1498. 1499. 1500. 1501. 1502. 1503. 1504. 1505. 1506. 1507. 1508. 1509. 1510. 1511. 1512. 1513. 1514. 1515. 1516. 1517. 1518. 1519. 1520. 1521. 1522. 1523. 1524. 1525. 1526. 1527. 1528. 1529. 1530. 1531. 1532. 1533. 1534. 1535. 1536. 1537. 1538. 1539. 1540. 1541. 1542. 1543. 1544. 1545. 1546. 1547. 1548. 1549. 1550. 1551. 1552. 1553. 1554. 1555. 1556. 1557. 1558. 1559. 1560. 1561. 1562. 1563. 1564. 1565. 1566. 1567. 1568. 1569. 1570. 1571. 1572. 1573. 1574. 1575. 1576. 1577. 1578. 1579. 1580. 1581. 1582. 1583. 1584. 1585. 1586. 1587. 1588. 1589. 1590. 1591. 1592. 1593. 1594. 1595. 1596. 1597. 1598. 1599. 1600. 1601. 1602. 1603. 1604. 1605. 1606. 1607. 1608. 1609. 1610. 1611. 1612. 1613. 1614. 1615. 1616. 1617. 1618. 1619. 1620. 1621. 1622. 1623. 1624. 1625. 1626. 1627. 1628. 1629. 1630. 1631. 1632. 1633. 1634. 1635. 1636. 1637. 1638. 1639. 1640. 1641. 1642. 1643. 1644. 1645. 1646. 1647. 1648. 1649. 1650. 1651. 1652. 1653. 1654. 1655. 1656. 1657. 1658. 1659. 1660. 1661. 1662. 1663. 1664. 1665. 1666. 1667. 1668. 1669. 1670. 1671. 1672. 1673. 1674. 1675. 1676. 1677. 1678. 1679. 1680. 1681. 1682. 1683. 1684. 1685. 1686. 1687. 1688. 1689. 1690. 1691. 1692. 1693. 1694. 1695. 1696. 1697. 1698. 1699. 1700. 1701. 1702. 1703. 1704. 1705. 1706. 1707. 1708. 1709. 1710. 1711. 1712. 1713. 1714. 1715. 1716. 1717. 1718. 1719. 1720. 1721. 1722. 1723. 1724. 1725. 1726. 1727. 1728. 1729. 1730. 1731. 1732. 1733. 1734. 1735. 1736. 1737. 1738. 1739. 1740. 1741. 1742. 1743. 1744. 1745. 1746. 1747. 1748. 1749. 1750. 1751. 1752. 1753. 1754. 1755. 1756. 1757. 1758. 1759. 1760. 1761. 1762. 1763. 1764. 1765. 1766. 1767. 1768. 1769. 1770. 1771. 1772. 1773. 1774. 1775. 1776. 1777. 1778. 1779. 1780. 1781. 1782. 1783. 1784. 1785. 1786. 1787. 1788. 1789. 1790. 1791. 1792. 1793. 1794. 1795. 1796. 1797. 1798. 1799. 1800. 1801. 1802. 1803. 1804. 1805. 1806. 1807. 1808. 1809. 1810. 1811. 1812. 1813. 1814. 1815. 1816. 1817. 1818. 1819. 1820. 1821. 1822. 1823. 1824. 1825. 1826. 1827. 1828. 1829. 1830. 1831. 1832. 1833. 1834. 1835. 1836. 1837. 1838. 1839. 1840. 1841. 1842. 1843. 1844. 1845. 1846. 1847. 1848. 1849. 1850. 1851. 1852. 1853. 1854. 1855. 1856. 1857. 1858. 1859. 1860. 1861. 1862. 1863. 1864. 1865. 1866. 1867. 1868. 1869. 1870. 1871. 1872. 1873. 1874. 1875. 1876. 1877. 1878. 1879. 1880. 1881. 1882. 1883. 1884. 1885. 1886. 1887. 1888. 1889. 1890. 1891. 1892. 1893. 1894. 1895. 1896. 1897. 1898. 1899. 1900. 1901. 1902. 1903. 1904. 1905. 1906. 1907. 1908. 1909. 1910. 1911. 1912. 1913. 1914. 1915. 1916. 1917. 1918. 1919. 1920. 1921. 1922. 1923. 1924. 1925. 1926. 1927. 1928. 1929. 1930. 1931. 1932. 1933. 1934. 1935. 1936. 1937. 1938. 1939. 1940. 1941. 1942. 1943. 1944. 1945. 1946. 1947. 1948. 1949. 1950. 1951. 1952. 1953. 1954. 1955. 1956. 1957. 1958. 1959. 1960. 1961. 1962. 1963. 1964. 1965. 1966. 1967. 1968. 1969. 1970. 1971. 1972. 1973. 1974. 1975. 1976. 1977. 1978. 1979. 1980. 1981. 1982. 1983. 1984. 1985. 1986. 1987. 1988. 1989. 1990. 1991. 1992. 1993. 1994. 1995. 1996. 1997. 1998. 1999. 2000. 2001. 2002. 2003. 2004. 2005. 2006. 2007. 2008. 2009. 2010. 2011. 2012. 2013. 2014. 2015. 2016. 2017. 2018. 2019. 2020. 2021. 2022. 2023. 2024. 2025. 2026. 2027. 2028. 2029. 2030. 2031. 2032. 2033. 2034. 2035. 2036. 2037. 2038. 2039. 2040. 2041. 2042. 2043. 2044. 2045. 2046. 2047. 2048. 2049. 2050. 2051. 2052. 2053. 2054. 2055. 2056. 2057. 2058. 2059. 2060. 2061. 2062. 2063. 2064. 2065. 2066. 2067. 2068. 2069. 2070. 2071. 2072. 2073. 2074. 2075. 2076. 2077. 2078. 2079. 2080. 2081. 2082. 2083. 2084. 2085. 2086. 2087. 2088. 2089. 2090. 2091. 2092. 2093. 2094. 2095. 2096. 2097. 2098. 2099. 2100. 2101. 2102. 2103. 2104. 2105. 2106. 2107. 2108. 2109. 2110. 2111. 2112. 2113. 2114. 2115. 2116. 2117. 2118. 2119. 2120. 2121. 2122. 2123. 2124. 2125. 2126. 2127. 2128. 2129. 2130. 2131. 2132. 2133. 2134. 2135. 2136. 2137. 2138. 2139. 2140. 2141. 2142. 2143. 2144. 2145. 2146. 2147. 2148. 2149. 2150. 2151. 2152. 2153. 2154. 2155. 2156. 2157. 2158. 2159. 2160. 2161. 2162. 2163. 2164. 2165. 2166. 2167. 2168. 2169. 2170. 2171. 2172. 2173. 2174. 2175. 2176. 2177. 2178. 2179. 2180. 2181. 2182. 2183. 2184. 2185. 2186. 2187. 2188. 2189. 2190. 2191. 2192. 2193. 2194. 2195. 2196. 2197. 2198. 2199. 2200. 2201. 2202. 2203. 2204. 2205. 2206. 2207. 2208. 22

场景一：

名称：用户乘电梯上楼

参与执行者实例：电梯控制器、用户、按钮、电梯、电梯门、马达

前置条件：电梯设备正常通电，系统已开机。

后置条件：系统处于就绪状态。

事件流：1) 用户在楼底层按动向上按钮，按动按钮事件将信号发送给电梯控制器，然后电梯控制器使电梯移动到底层，然后打开电梯门。

2) 用户按楼层按钮，按动事件将发送给电梯控制器，电梯控制器等待超时，关闭门，控制马达向上升起并等待到达事件，当电梯到达用户请求的楼层后，电梯控制门打开，在动作序列中，电梯控制器与门对象和马达对象交互。

场景二：

名称：用户乘电梯下楼

参与执行者实例：电梯控制器，用户，按钮，电梯，电梯门，马达；

前置条件：电梯设备正常通电，系统已开机

后置条件：系统处于就绪状态。

事件流：1) 用户在除底层外的楼层，按动电梯等待室里的向下按钮召唤电梯并等待。电梯移至用户所在的楼层。

2) 电梯停止在用户所在楼层后打开门，用户走进电梯并按下电梯内部控制面板上的底层按钮。

3) 电梯门关闭，下降并停在底层，电梯打开，等待片刻，然后关门。

第七章

为什么需要需求规格说明？结合实验进行说明。

- 1) 需求规格说明描述了软件系统的解决方案，书写需求规格说明，可以建立管理控制的基线，方便任务分配，制定工作计划，并进行跟踪、度量。
 - 2) 方便交流，软件需求是项目中需要进行广泛交流的内容之一，文档化有利于沟通交流。用户需要验证文档内描述的需求信息与其最初意图一致；项目管理者基于它进行软件估算，并根据估算数据安排进度和人员分工；设计人员和程序员需要依据需求文档来完成自己的任务，依次判断其工作是否正确；测试人员，根据文档的需求内容进行验收测试，确保产品符合用户要求；文档编写人员依此编写用户手册；维护人员依此理解原有软件。对给定的需求示例，判断并修正其错误。在实验中，以需求为基础进行了系统的功能测试，开发测试用例套件，在系统级需求上达到测试全覆盖。并且后续每个阶段几乎都是以需求规格为依据，尤其是在设计阶段评审的重要内容就是是否符合需求规格。
 - 3) 在实验中，需求规格的重要性不只体现在结果上，还包括中间过程，在书写需求规格过程中，才真正把问题域的问题和分析模型的成果转化为系统级需求，方便小组成员真正明确需求，个人认为在这个阶段包含一部分的需求在发现和完整化。
- 1) 需求规格说明描述了软件系统的解决方案，书写需求规格说明，可以建立管理控制的基线，方便任务分配，制定工作计划，并进行跟踪、度量
- 2) 方便交流

软件开发的子任务与人员之间存在着错综复杂的关系，存在大量的沟通 and 交流，所以软件系统开发中需要编写多种不同类型的文档，每种文档都针对项目中需要进行广泛交流的内容。软件需求是项目中需要进行广泛交流的内容之一，所以需求开发阶段需要进行需求的文档化。

为给定的需求示例，判定并修正其错误

对给定的需求规格说明片段，找出并修正其错误

1.技术文档写作要点：

简洁

精确

易读（查询）

易修改

2.需求书写要点：

使用用户术语（不要使用“计算机术语”，如函数、对象、参数、类等）

可验证（让需求具体化，小心形容词和副词的使用，避免程度词的使用）P120

可行性（理论上的技术实现可能性，限定的成本、时间和人员约束内）

为给定的需求示例，设计功能测试用例 P125

结合测试方法

ID	输入				预期输出
	商品信息	特价	赠品	支付	
TUSI-1	无商品	无	无	无	不做任何处理，关闭销售任务
TUSI-2	商品1（1、1(双)、35) 商品2（2、1(双)、50)	无	无		
TUSI-3	商品1（1、1(双)、35) 商品2（2、1(双)、50)	无	无	85	无找零，系统行为满足后置条件
TUSI-4	商品1（1、1(双)、35) 商品2（2、1(双)、50)	商品1特价20		100	找零30，系统行为满足后置条件
TUSI-5	商品1（1、1(双)、35) 商品2（2、1(双)、50)	总额特价50以上0.8折		100	找零31，系统行为满足后置条件
TUSI-6	商品1（1、1(双)、35) 商品2（2、1(双)、60)	商品1特价20 总额特价50以上0.8折		100	找零32，系统行为满足后置条件

第八章

名词解释

软件设计

软件设计是指关于软件对象的设计，是一种设计活动。软件设计既指软件对象实现的规格说明，又指这个规格说明产生的过程。

首先进行概要设计，根据功能设计软件的整体模块结构；其次进行详细设计，建立模块的层次化分解，定义高质量、可实现的细化模块，并设计各细化模块内部的程序结构。

软件设计的核心思想是什么？

抽象和分解是软件设计的核心思想。

分解：横向上将系统分解为几个相对简单的子系统以及各子系统之间的关系

抽象：纵向上聚焦各子系统的接口（区别于实现，各子系统之间交互的契约），可以分离接口和实现，使得人们更好地关注软件系统本质，降低复杂度

软件工程设计有哪三个层次？各层次的主要思想是什么？

高层设计：基于反映软件高层抽象的构建层次，描述系统的高层结构、关注点和设计决策

中层设计：更加关注组成构件的模块的划分、导入/导出、过程之间的调用关系或者类之间的协作

低层设计：深入模块和类的内部，关注具体的数据结构、算法、类型、语句和控制结构等

第九、十章

体系结构的概念

一个软件系统的体系结构规定了系统的计算部件和部件之间的交互

体系结构的风格的优缺点

1.主程序/子程序

优点：

- 1) 流程清晰，易于理解
- 2) 强控制性

缺点：

1) 程序调用是一种强耦合的连接方式，非常依赖交互方的接口规格，这会使得系统难以修改和复用

2) 程序调用的连接方式限制了各部件之间的数据交互，可能会使得不同部件使用隐含的共享数据交流，产生不必要的公共耦合，进而破坏它的“正确性”控制能力

2.面向对象式

优点：

- 1) 内部实现的可修改性
- 2) 易开发、易理解、易复用的结构组织

缺点：

- 1) 接口的耦合性
- 2) 标识的耦合性
- 3) 副作用 P155

3.分层

优点：

- 1) 设计机制清晰，易于理解
- 2) 支持并行开发
- 3) 更好的可复用性与内部可修改性

缺点：

- 1) 交互协议难以修改
- 2) 性能损失
- 3) 难以确定层次数量和粒度

4.MVC

优点：

- 1) 易开发性
- 2) 视图和控制的可修改性

3) 适宜于网络系统开发的特征

缺点:

- 1) 复杂性
- 2) 模型修改困难

体系结构设计的过程?

- 1) 分析关键需求和项目约束
- 2) 选择体系结构风格
- 3) 进行软件体系结构逻辑(抽象)设计
- 4) 依赖逻辑设计进行软件体系结构物理(实现)设计
- 5) 完善软件体系结构设计
- 6) 定义构件接口
- 7) 迭代过程 3)~6)

包的原则

- Reuse-Release Equivalency Principle
 - Common Closure Principle
 - Common Reuse Principle
 - Acyclic Dependencies Principles
 - Stable Dependencies Principles
 - Stable Abstractions Principles
- Cohesion
Reuse &&
Change
- Coupling
Compile
&&
Link

- 1) 重用发布等价原则 (REP): 重用的粒度就是发布的粒度
- 2) 共同封闭原则 (CCP): 包中所有类对于同一类性质的变化应该是共同封闭的。一个变化若对一个包产生影响, 则对该包中所有类产生影响, 而对于其他包不造成任何影响。
- 3) 共同重用原则 (CRP): 一个包中的所有类应该是共同重用的。
- 4) 无环依赖原则 (ADP): 在包的依赖关系图中不能存在环
- 5) 稳定依赖原则 (SDP): 朝着稳定的方向进行依赖
- 6) 稳定抽象原则 (SAP): 包的抽象程度应该和其稳定程度一致

体系结构构件之间接口的定义 (*)

提供的服务 (供接口)		
LoginController.login	语法	public ResultMessage login(long id, String password);
	前置条件	用户输入有效的用户编号和密码。user 成员变量初始化为 null。
	后置条件	根据 id 查找是否存在相应的用户, 根据用户信息验证其密码正确性, 如果通过验证记录当前用户。
需要的服务 (需接口)		
服务名	服务	
UserDataService.find	根据 ID 进行查找用户信息	

体系结构开发集成测试用例 P177

第十一章

名词解释

可用性

易用性是人机交互中一个既重要又复杂的概念。它不仅关注人使用系统的过程，同时还关注系统对使用它的人所产生的作用。易用性是多维度的质量属性，维度包括：易学性、易记性、效率、出错率和主观满意度。

能够列出至少 5 个界面设计的注意事项，并加以解释

例子**违反了哪些界面设计原则

1.简洁设计

2.一致性设计

3.低出错率设计

4.易记性设计

减少短期记忆负担

使用逐层递进的方式展示信息

使用直观的快捷方式

设置有意义的默认值

5.可视化设计要点

按照任务模型设计界面隐喻，同时不要把软件系统的内部构造机制暴露给用户

可视化设计还应该基于界面隐喻，尽可能地把功能和任务细节表现出来



分析上图：它们分别体现（或违反）了哪些人机交互设计原则？请详细解释这些原则（10分）。

违反了：

一致性设计：不一致的交互机制会导致用户的精神模型的不一致，造成不必要的麻烦和负担。

体现了：

反馈：对用户行为进行了反馈，让用户意识到行为的结果，单击按钮改变了按钮边框。

简洁设计：第二张图隐喻设计代替描述文字
可视化设计：基于界面隐喻，把功能和任务细节表现了出来。



该界面很好的运用了哪些人机交互原则？请至少列出3条并进行详细的解释

导航：能帮助用户找到任务的入口
简洁设计
易记性设计：使用直观的快捷方式

精神模型、差异性

精神模型就是用户进行人机交互时头脑中的任务模型。人机交互设计需要依据精神模型进行隐喻设计

对不同用户群体的任务模型是有差异的，所以对他们的人机交互设计也要有差异。按照用户群体自身的特点，可以将其划分为新手用户、专家用户和熟练用户

导航、反馈、协作式设计

软件系统**导航**要能帮助用户找到任务的入口，导航的目的就是为用户提供一个很好的完成任务的入口，好的导航会让这个入口非常符合人的精神模型

好的人机交互设计需要对用户行为进行**反馈**，让用户能够意识到行为的结果。反馈的目的是提示用户交互行为的结果，但不能打断用户工作时的意识流。对用户思考和反应时间的把握。人和计算机是人机交互的两方，其中人的因素是比较固定的，一定时期内不会发生大的变化，所以要让两者交互顺畅，就需要让计算机更多地适应人的因素，这也是人机交互设计以用户为中心的根本原因。这种调整计算机因素以更好地适应并帮助用户的设计方式被称之为**协作式设计**。

第十二章

详细设计的出发点

以需求开发的结果（需求规格说明书和需求分析模型）和软件体系结构的结果（软件体系结

构设计方案与原型)为出发点。

在详细设计中一般进行中层设计和低层设计,详细设计的目的是实现所有功能性需求和非功能性需求

职责分配

Responsibility assignment :

- Some responsibilities are from RE(SRS)
 - Typically use cases, domain models, sequence diagrams, state diagrams
- Some others are from implementation decisions

• Problem:

- What is the most basic principle by which responsibilities are assigned in object-oriented design?

• Solution:

- Assign a responsibility to the class that has the information necessary to fulfill the responsibility.

- 1) 职责是持有某项数据或者表现某种职责的义务;数据职责主要由属性满足,行为职责主要由方法满足;可能会包含类间协作
- 2) 职责可以从不同的抽象层次开始;职责是可以被分解的;职责分解可以是分解组件的基础
- 3) 职责分配可以帮助实现高内聚低耦合,确保不同模块职责没有重合,当且仅当数据和方法有助于实现模块职责时将其添加给模块。
- 4) GRASP 原则(通用职责分配软件模式—将职责分配原则总结为模式):

① Expert 专家: 将一个职责分配给专家—掌握了履行职责所必需的信息的类。

② Creator 创建者:

将创建一个类 A 的实例的职责指派给类 B 的实例,如果下列条件满足的话:

■ B 聚集了 A 对象。

■ B 包含了 A 对象。

■ B 记录了 A 对象的实例。

■ B 要经常使用 A 对象。

■ 当 A 的实例被创建时, B 具有要传递给 A 的初始化数据(也就是说 B 是创建 A 的实例这项任务的信息专家)。

B 是 A 对象的创建者。

如果以上的条件中不止一条成立的话,那么最好是让 B 聚集或者包含 A。

③ 高内聚: 分配一个职责的时候要保持类的高聚合度

④ 低耦合: 分配一个职责是要保持低耦合度

⑤ Controller:

- 1.The business or overall organization (a façade controller).
- 2.The overall "system" (a façade controller).
- 3.An animate thing in the domain that would perform the work (a role controller).
- 4.An artificial class (Pure Fabrication) representing the use (a use case controller).

控制要点:

- A) 避免大多数消息由一个类发出
- B) 组件相对较小
- C) 行为、职责和数据绑定
- D) 职责单一

协作

抽象对象之间的协作

- 1.从小到大,将对象的小职责聚合形成大职责;
- 2.从大到小,将大职责分配给各个小对象。
- 这两种方法,一般是同时运用的,共同来完成对协作的抽象。

控制风格

1) 集中式：做决策的只有一个对象，

Centralized Control

- Easy to find where decisions are made
- Easy to see how decisions are made and to alter the decision-making process
- Controllers may become bloated—large, complex, and hard to understand, maintain, test, etc.
- Controller may treat other components as data repositories
 - Increases coupling
 - Destroys information hiding

2) 委托式：作出决策的对象不只有一个，职责的分解决定了控制对象的层次。

3) 分散式：无法找到明确的控制对象，每个对象都只承担一个相对较小的职责，完全靠对象自治的方式实现各自的职责。

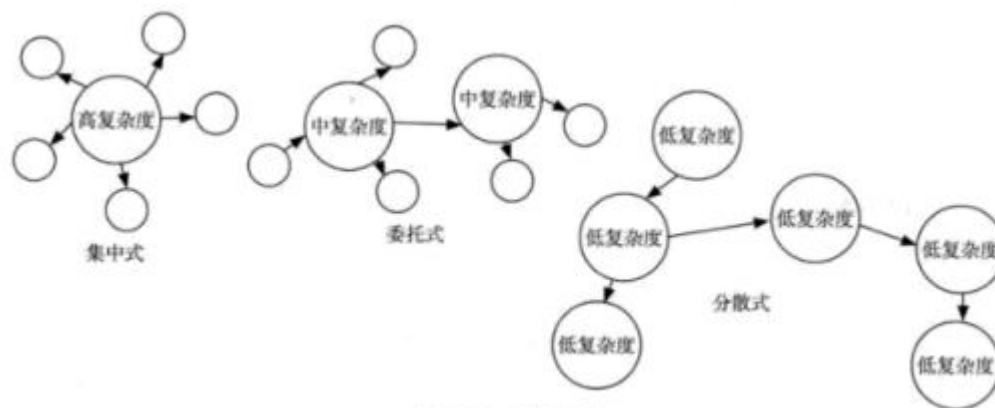


图 12-13 控制风格

给定分析类图、系统顺序图和设计因素描述

建立设计类图 P205

或者详细顺序图 P207

协作的测试

MockObject P211

对例子**，说明它们之间的耦合程度与内聚，给出理由

Code Snippet 1

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i.name)) {
        error_message("Invalid name");
    }

    if (!valid_string(i.book)) {
        error_message("Invalid book name");
    }

    if (!valid_month(i.date)) {
        error_message("Invalid month");
    }
}

int valid_month(date d)
{
    return d.month >= 1 && d.month <= 12;
}
```

通信内聚、数据耦合

Code Snippet 2

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i)) {
        error_message("Invalid name");
    }

    if (!valid_string(i)) {
        error_message("Invalid book name");
    }

    if (!valid_month(i)) {
        error_message("Invalid month");
    }
}

int valid_month(input_form i)
{
    return i.date.month >= 1 && i.date.month <= 12;
}
```

印记耦合

Code Snippet 3

```
void validate_checkout_request(input_form i)
{
    if (!valid(i.name, STRING)) {
        error_message("Invalid name");
    }

    if (!valid(i.book, STRING)) {
        error_message("Invalid book name");
    }

    if (!valid(i.date, DATE)) {
        error_message("Invalid month");
    }
}

int valid(string s, int type)
{
    switch (type) {
        case STRING:
            return strlen(s) < MAX_STRING_SIZE;
        case DATE:
            date d = parse_date(s);
            return d.month >= 1 && d.month <= 12;
    };
}
```

控制耦合

Code Snippet 4

```
string patron_name, book_name;
date checkout_date;

void validate_checkout_request(input_form i)
{
    patron_name = i.name;
    if (!valid_string()) {
        error_message("Invalid name");
    }

    book_name = i.book;
    if (!valid_string()) {
        error_message("Invalid book name");
    }

    checkout_date = i.date;
    if (!valid_month()) {
        error_message("Invalid month");
    }
}

int valid_month()
{
    return checkout_date.month >= 1 && checkout_date.month <= 12;
}
```

公共耦合

Code Snippet 5

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i.name)) {
        i.string = "Invalid name";
        error_message();
    }

    if (!valid_string(i.book)) {
        i.book = "Invalid book name";
        error_message();
    }

    valid_month(i.date);
}

void valid_month(date d)
{
    if (d.month < 1) {
        d.month = 1;
    }

    if (d.month > 12) {
        d.month = 12;
    }

    return 1;
}
```

内容耦合

Code Snippet 6 -- 重复耦合

```
void validate_checkout_request(input_form i)
{
    int len = 0;
    boolean valid_string = false;

    len = i.name.length();
    char arr1[] = new char[len];
    for(char c: arr1){
        if c是小写字母 valid_string = true;
    }
    if (!valid_string) {
        error_message("Invalid name");
    }

    len = i.book.length();
    char arr2[] = new char[len];
    for(char c: arr2){
        if c是小写字母 valid_string = true;
    }
    if (!valid_string) {
        error_message("Invalid book name");
    }

    if (!valid_month(i.date)) {
        error_message("Invalid month");
    }
}
```

```
Public class Rous{

    public static int findPattern( String text, String pattern)
    { ... }
    public static int average( Vector numbers )
    { ... }
    public static OutputStream openFile( String fileName )
    { ... }
}
```

偶然内聚

```

public void sample( String flag ) {
    switch ( flag ){
        case ON:
            .....
            break;
        case OFF:
            .....
            break;
        case CLOSE:
            .....
            break;
    }
}

```

逻辑内聚

```

Public class foo {
    Private string name;
    Private int size;
    //constructor
    public void foo(){

        this.name = "Not Set";
        this.size = 12;
    }
    //destructor
    Public void ~foo() {
        delete[] name;
        delete size;
    }
}

```

时间内聚


```

void MonthEnd()
{
    Report ExR = InitExpenseReport();
    Report rr = InitRevenueReport();
    Report EmpR = InitEmployeeReport();

    EmpR.Init();
    rr.Init();
    ExR.SetEmployees(true);

    if (ExR.GetReportParams())
        EmpR.GetReportParams();

    SendToPrinter(rr);
    SendToPrinter(ExR);
    SendToPrinter(EmpR);
}

```

过程内聚

```

Public class Calculate {
    Public int product;
    public void product(int a, int b){
        product = a*b;
        ....
        save(product);
        ....
    }
    Public void save(int product) {
        \\ code to store value into database
        ...
    }
}

```

通信内聚

```
public int commission(int sale, long percentage){  
    int com;  
    //calculate commission  
    return com;  
}
```

功能内聚

```
Public interface Addressee{  
    .....  
    Public abstract String getName ();  
    Public abstract String getAddress ();  
    ....  
}  
  
Public class Employee implements Addressee {....}
```

信息内聚

```

public class Vector3D{
    public int x, y, z;
    ...
}
public class Arch {
    private Vector3D baseline;
    ...
    void slant(int newY){
        baseline.x = 10;
        baseline.y = 13;
    }
}

```

内容耦合

```

public routineX(String command) {
    if (command.equals("drawCircle")){
        drawCircle();
    }
    else{
        drawRectangle();
    }
}

```

控制耦合

```

public class Employee{
    public String name, emailID.
    ...
}

public class Emailer{
    public void sendEmail(Employee e, String text)
    {...}
    ...
}

```

印记耦合

```

Public class Receiver {
    public void message( MyType X ){
        ...
        X.doSomethingForMe( Object data );
        ...
    }
}

```

数据耦合

```

int x;

public class myValue{
    public void addValue(int a){
        x = x+ a;
    }
    public void subtractValue(int a){
        x = x- a;
    }
}

```

公共耦合

信息隐藏

基本思想

信息隐藏利用了抽象的方法，抽象出每个类的关键细节，也就是模块的职责。换句话说，抽象出来的就是接口，隐藏的就是实现，他们共同体现了模块的职责。

信息隐藏的核心设计思路是每个模块都隐藏一个重要的设计决策。每个模块都承担一定的职责，对外表现为一份契约，并且在这份契约之下隐藏着只有这个模块知道的设计决策或者秘密，决策实现的细节（特别是容易改变的细节）只有该模块自己知道。

两种常见的信息隐藏决策

“职责的实现”和“实现的变更”

对例子**，说明其信息隐藏程度好坏

模块易变化的地方：根据需求分配的职责，内部实现机制

第十四章

- Principles from Modularization
 - 1: 《Global Variables Consider Harmful》
 - 2: 《To be Explicit》
 - 3: 《Do not Repeat》
 - 4: Programming to Interface (Design by Contract)
 - 5: The Law of Demeter
 - 6: Interface Segregation Principle (ISP)
 - 7: Liskov Substitution Principle (LSP)
 - 8: Favor Composition Over Inheritance
 - 9: Single Responsibility Principle
- 对给定的示例，发现其所违反的原则，并进行修正

- ① 全局变量是有害的
- ② 简洁
- ③ 不要重复
- ④ 针对接口编程
- ⑤ 迪米特法则
- ⑥ 接口最小化原则
- ⑦ Liskov 替换原则
- ⑧ 多用组合少用继承
- ⑨ 单一职责原则

降低访问耦合的方法

1. 针对接口编程

2. 接口最小化/接口分离原则

3.访问耦合的合理范围、迪米特法则

降低继承耦合的方法

1.LSP: Liskov 替换原则

子类型必须能够替换掉基类型而起同样的作用。

即：子类方法的前置条件必须与超类方法的前置条件相同甚至要求更少；子类方法的后置条件必须与超类方法的后置条件相同或者要求更多

2.使用组合代替继承

利用组合关系，既能复用代码，又能保持接口的灵活性

在希望复用代码又不能满足 LSP 时，往往会用组合来替代继承

提高内聚的方法

1.集中信息与行为

2.SRP: 单一职责原则

第十五章

信息隐藏的含义

面向对象中，需要

- 1) 封装类的职责，隐藏职责的实现
- 2) 预计将会发生的变更，抽象它的接口，隐藏内部实现机制

封装

分离接口与实现

- 1) 将数据和行为同时包含在类中
- 2) 分离对外接口和内部实现

实现细节

- 1.封装数据与行为
- 2.封装内部结构
- 3.封装其他对象的引用

```
public Position getPosition( int index )
{
    // 重新构造了一个对象返回，隐藏了实现细节
    Position position = new Position( positions.get( index ) );
    return position;
}
```

4. 封装类型信息

5. 封装潜在变更

OCP

开闭原则

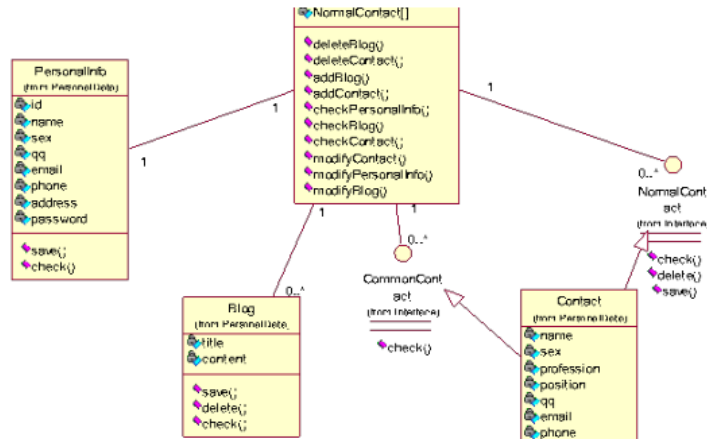
- 1) 好的设计应该对“扩展”开放
- 2) 好的设计应该对“修改”关闭

简单来说，开闭原则是指：在发生变更时，好的设计只需要添加新的代码而不需要修改原有的代码，就能够实现变更

DIP

依赖倒置原则

抽象不应该依赖于细节，细节应该依赖于抽象。因为抽象是稳定的，细节是不稳定的
高层模块不应该依赖于低层模块，而是双方都依赖于抽象。因为抽象是稳定的，而高层模块和低层模块都可能是不稳定的



分析下图请说明：（1）它正确运用了哪条面向对象设计原则，请详细解释这条原则（5分）；（2）User类的内部是哪种层次的内聚？请详细描述这个层次内聚的特征（5分）；（3）它违反了哪条面向对象设计原则，请详细解释这条原则（5分）。

第十六章

如何实现可修改性、可扩展性、灵活性

通过接口与实现的分离来实现。

- 1) 通过接口和实现该接口的类完成接口与实现的分离
- 2) 通过子类继承父类，将父类的接口和子类的实现相分离

策略模式

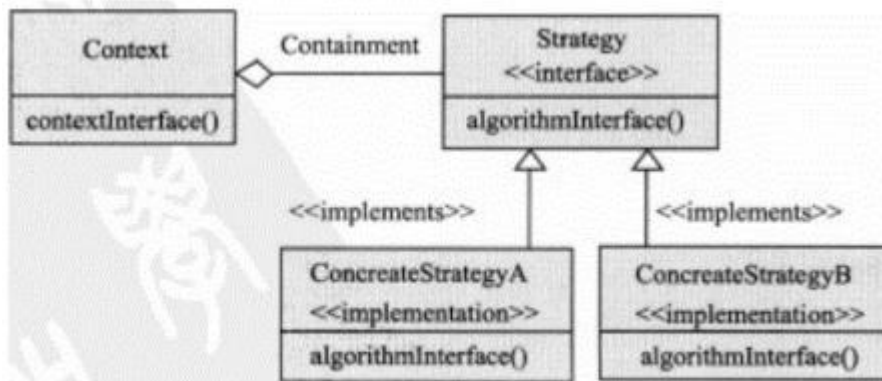


图 16-8 策略模式

抽象工厂模式

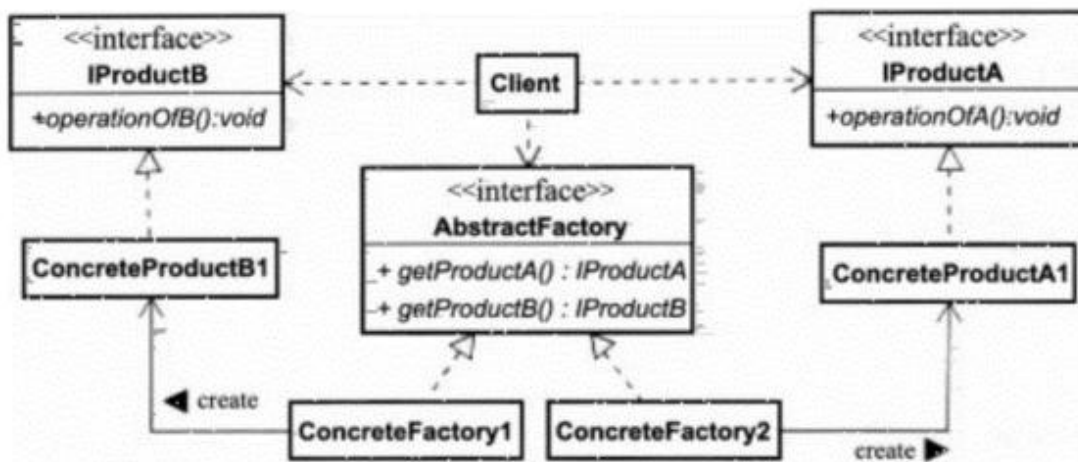
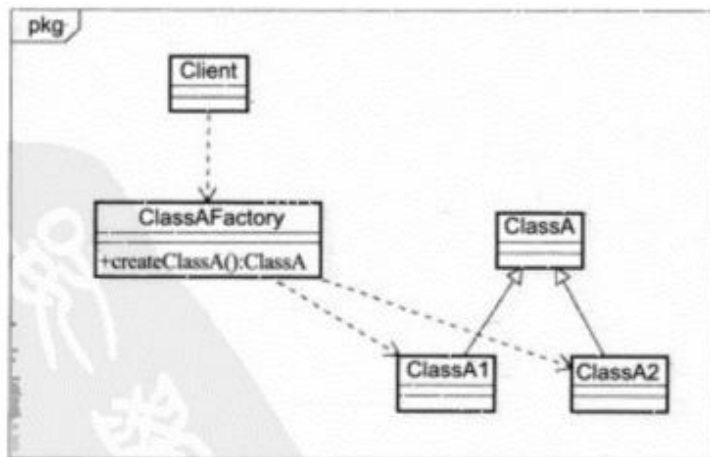
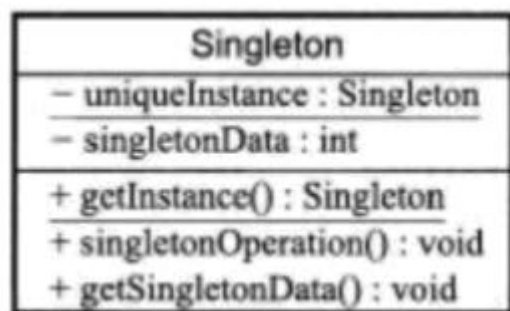


图-16-15 抽象工厂模式

单件模式



```

public class DatabaseFactoryTxtFileImpl implements DatabaseFactory {
    //单键
    private static DatabaseFactoryTxtFileImpl databaseFactoryTxtFile = null;

    private DatabaseFactoryTxtFileImpl(){

    }

    public static DatabaseFactory getInstance(){
        if(databaseFactoryTxtFile==null)
            databaseFactoryTxtFile=new DatabaseFactoryTxtFileImpl();
        return databaseFactoryTxtFile;
    }
}

```

迭代器模式

给定场景，应用设计模式并写出代码

给出代码，要求用设计模式改写

第十七、十八章

构造包含的活动

定义问题，需求分析，规划构建，软件架构，详细设计，编码与调试，集成，单元测试，集成测试，系统测试，保障维护

名词解释

重构

修改软件系统的严谨方法，它在不改变代码外部表现的情况下改进其内部结构。不改变代码的外部表现，是指不改变软件系统的功能。改进代码的内部结构是指提升详细设计结构的质量，使其能够继续演化下去。

测试驱动开发

测试优先的开发

结对编程

两个程序员挨着坐在一起，共同协作进行软件构造活动

给定代码段示例，对其进行改进或者发现其中的问题

简洁性/可维护性

使用数据结构消减复杂判定

控制结构

变量使用

语句处理

How to write unmaintainable code

防御与错误处理

单元测试用例的设计

契约式设计

如果一个函数或方法，在前置条件满足的情况下开始执行，完成后能够满足后置条件，那么这个函数或方法就是正确、可靠的。常见编程方式：异常与断言

防御式编程

在一个方法或其他方法、操作系统、硬件等外界环境交互时，不能确保外界都是正确的，所以要在外界发生错误时，保护方法内部不受损害

表驱动 P307

```
prePointArray = { 1000, 2000, 5000 };
postPointArray = { 1000, 2000, 5000 };
levelArray = { 1, 2, 3 };
for (int i=0; i<=2; i++) {
    if ( (prePoint < prePointArray[i]) && (postPoint >= postPointArray[i])) {
        triggerGiftEvent (levelArray[i]);
    }
}
```

练习 - 表驱动

- For the first \$10,000 of income, the tax is 10%
- For the next \$10,000 of income above \$10,000, the tax is 12 percent
- For the next \$10,000 of income above \$20,000, the tax is 15 percent
- For the next \$10,000 of income above \$30,000, the tax is 18 percent
- For any income above \$40,000, the tax is 20 percent

- Define a tax table for each “bracket” of tax liability

Bracket	Base	Percent
0	0	10
10,000	1000	12
20,000	2200	15
30,000	3700	18
40,000	55000	20

- Simplified algorithm


```
for (int i=2; level=1; i <= 5; i++)
    if (taxable_income > bracket[i])
        level = level + 1;
tax= base[level]+percent[level] * (taxable_income - bracket[level]);
```

第十九章

掌握白盒测试和黑盒测试的常见方法，并进行优缺点比较

能解释并区别白盒测试三种不同的方法：语句覆盖、分支覆盖和路径覆盖

基于规格的技术——黑盒测试方法

把测试对象看做一个黑盒子，完全基于输入和输出数据来判定测试对象的正确性

1.等价类划分

2.边界值分析

3.决策表

4.状态转换

基于代码的技术——白盒测试方法

将测试对象看做是透明的，不关心测试对象的规格，而是按照测试对象内部的程序结构来设计测试用例进行测试工作

1.语句覆盖（对简单情况）

确保被测试对象的每一行程序代码都至少执行一次

2.分支覆盖（对复杂代码）

确保程序中每个判断的每个结果都至少满足一次

3.路径覆盖（对关键、复杂的代码）

确保程序中每条独立的执行路径都至少执行一次

给出一个场景，判断应该使用哪种测试方法，如何去写

对给定的场景和要求的测试方法，设计测试用例

给出功能需求，则要求写功能测试用例

ID	输入				预期输出
	商品信息	特价	赠品	支付	
TUSI-1	无商品	无	无	无	不做任何处理，关闭销售任务
TUSI-2	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	无	无		
TUSI-3	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	无	无	85	无找零，系统行为满足后置条件
TUSI-4	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	商品1特价20		100	找零30，系统行为满足后置条件
TUSI-5	商品1 (1、1(双)、35) 商品2 (2、1(双)、50)	总额特价50以上0.8折		100	找零31，系统行为满足后置条件
TUSI-6	商品1 (1、1(双)、35) 商品2 (2、1(双)、60)	商品1特价20 总额特价50以上0.8折		100	找零32，系统行为满足后置条件

给出设计图，则要求写集成测试用例，Stub and Driver
给出方法的描述，则要求写单元测试用例，Mock Object
JUnit 基本使用方法

第二十、二十一章

如何理解软件维护的重要性？

人们需要经常“修改”软件，修改软件的代价是非常高的，这使得软件维护将其工作重点放在了软件修改和变更上。

开发可维护软件的方法

分析需求的易变性

为变更进行设计

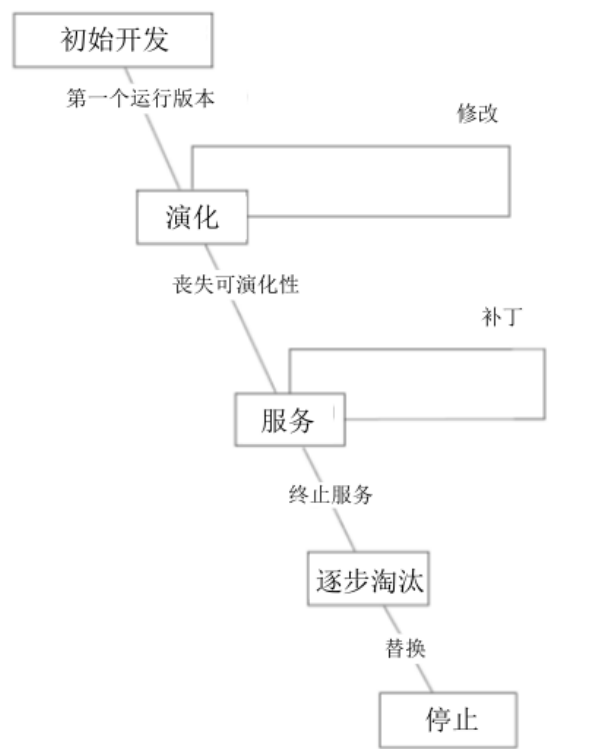
编写详细的技术文档并保持及时更新

保证代码的可读性

维护需求跟踪链

维护回归测试基线

演化式生命周期模型



软件演化生命周期模型

用户文档、系统文档

用户文档是指为用户编写参考指南或者操作教程，常见的有用户使用手册、联机帮助文档等与用户文档注重系统使用细节不同，系统管理员文档更注重系统维护方面的内容

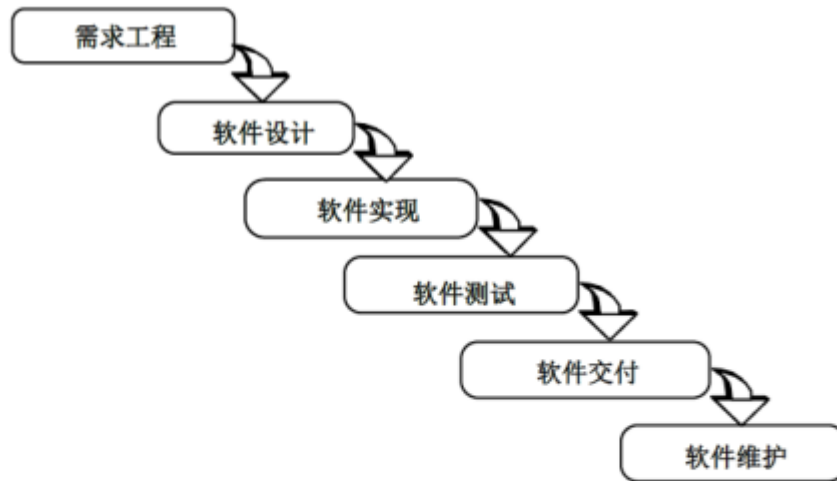
逆向工程、再工程

逆向工程技术是指：“分析目标系统，标识系统的部件及其交互关系，并且使用其他形式或者更高层的抽象创建系统表现的过程”

逆向工程的主要关注点是理解软件，但并不修改软件。再工程恰恰相反，它主要关注如何修改软件，不会花费很大气力理解软件。再工程的目的是对遗留软件系统进行分析 and 重新开发，以便进一步利用新技术来改善系统或促进现存系统的再利用。

第二十二、二十三章

软件生命周期模型



典型的软件生命周期

- Build-and-fix model
- Waterfall model
- Iterative Models
 - Incremental model
 - Incremental Delivery
 - Evolutionary
 - Evolutionary development
 - Prototyping
 - Spiral Model

解释与比较不同过程模型（要求、特征描述、优点、缺点）

构建-修复模型

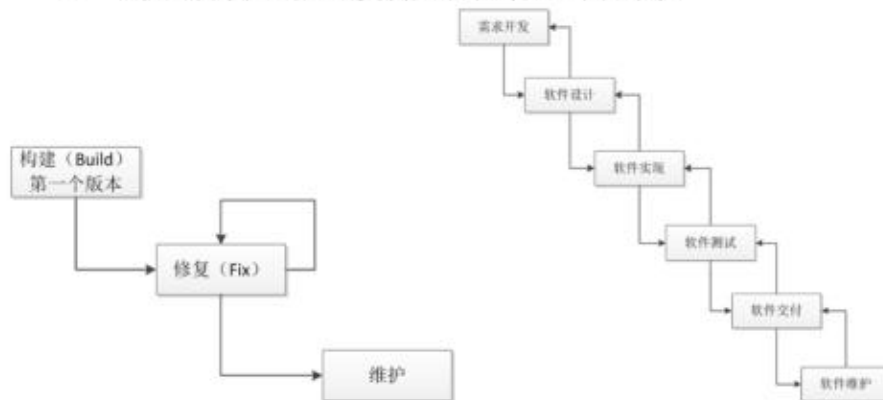
a) 构建修复模型-是最早也是最自然产生的软件开发模型。没有规划组织，完全依靠开发人员的个人能力。

i. 缺点：

1. 没有分工，随着软件系统复杂度上升，开发活动超出个人控制能力。
2. 没有分析需求的真实性，给软件开发带来很大的风险
3. 软件结构的质量随着修改越来越差
4. 没有考虑可维护性

ii. 使用范围：

1. 软件规模很小，只有几百行程序。
2. 对软件质量要求不高，即使出错也无所谓
3. 只关注开发，对后期维护要求不高甚至不需维护



瀑布模型

b) 瀑布模型：按照软件生命周期模型将软件开发活动组织为需求开发、软件设计、软件实现、软件测试、软件交付和软件维护等基本活动，并且规定了它们自上而下、相互邻接的次序。允许反复和迭代，重点在于每个活动必须进行验证，文档驱动的。

i. 缺点：

1. 对文档期待过高
2. 对开发活动的线性顺序假设是不符合实际情况的
3. 客户用户参与不够，只有在需求一个阶段用户参与
4. 里程碑粒度过粗，现在软件系统复杂，每个阶段持续周期较长。

ii. 优点：

1. 为软件开发活动定义了清晰的阶段划分（包括了输入/输出、主要工作及其关注点），降低了复杂度。

iii. 使用范围

1. 需求非常成熟稳定。例：开发一个电梯调度系统，需求是明确的。
2. 所需的技术成熟、可靠。例：开发一套自动化办公系统。
3. 复杂度适中，不至于产生太大的文档负担和过粗的里程碑。

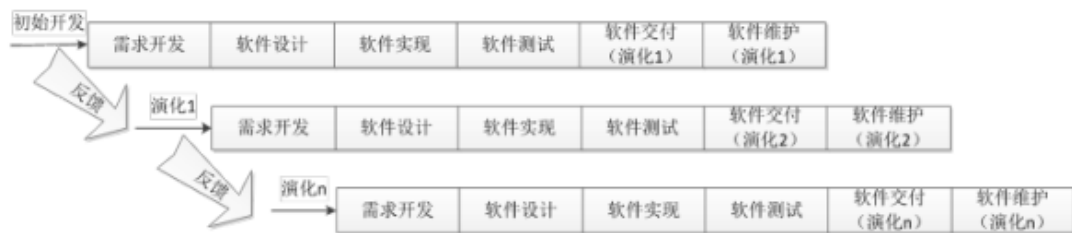
Note: 周期过长和渐进演化。时间压力和并行开发

增量迭代模型



- c) 增量迭代模型：项目开始时，通过系统需求的开发和核心体系结构设计活动完成项目前景和范围的商定，然后将后续开发活动组织为多个迭代、并行的瀑布式开发活动。**需求驱动的。**
- i. 优点：
 1. 迭代式开发符合软件开发的实际情况，更好的适用性。
 2. 并行开发可以缩短产品的开发时间
 3. 渐进交付可以加强用户反馈，降低开发风险。
 - ii. 缺点：
 1. 由于各个构件是逐渐并入已有的软件体系结构中的，所以加入构件必须不破坏已构造好的系统部分，要求具有开放式的体系结构。
 2. 增量交付模型需要一个完备、清晰的项目前景和范围以进行并发开发规划，但是在一些不稳定的领域，不确定性太多或者需求变化非常频繁，很难在项目开始就确定前景和范围
 - iii. 适用范围
 1. 大规模软件开发，比较成熟和稳定的领域。**如：电子商务。CSS 选课系统可以考虑。**

演化模型



d) 演化模型：演化模型将软件开发活动组织为多个迭代、并行的瀑布式开发活动。

i. 优点：

1. 迭代开发，尤其适用于需求变更频繁或不确定性较高的系统。
2. 并行开发
3. 渐进交付

ii. 缺点：

1. 无法在项目早期阶段确定项目范围，项目的整体计划，商务协商难以把握。
Manager 难度
2. 体系结构难做，体系结构几乎确定会发生变更。
3. 容易退化为构建修复方式。

iii. 使用范围：

1. 不稳定领域的大规模软件系统。如：互联网金融，大数据。CSS 选课系统
不适合，需求不易变更，体系结构难做。

原型模型



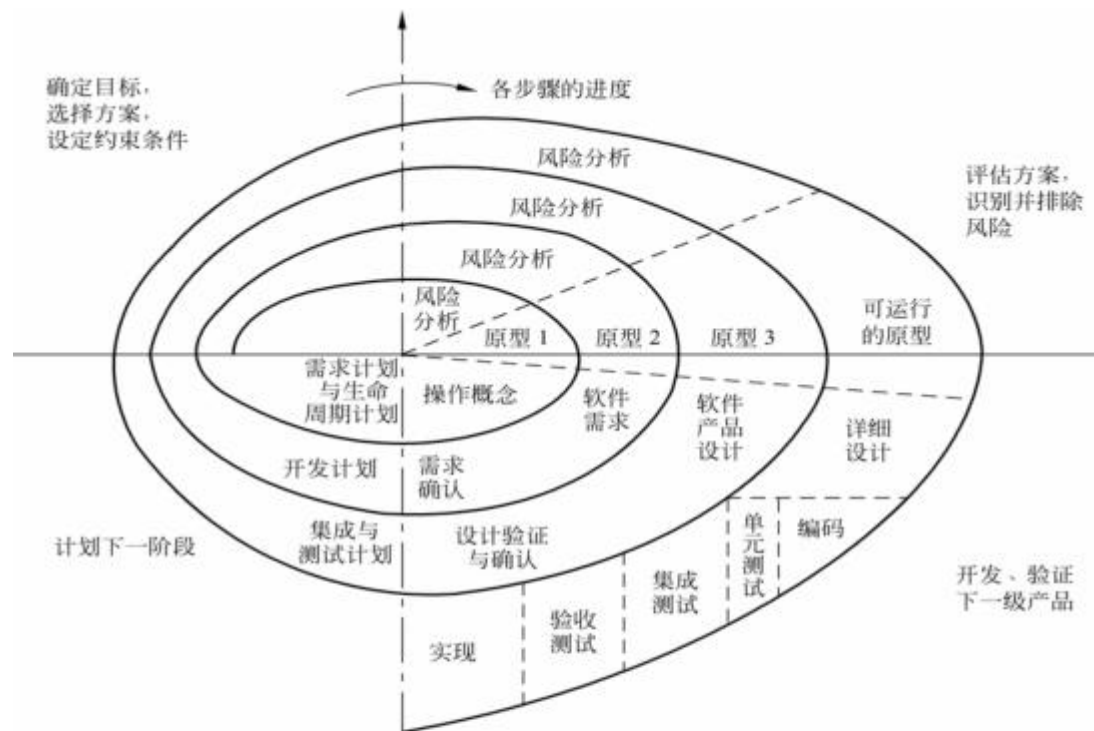
e) 原型模型：在整体安排迭代的情况下，强调“抛弃式原型”的演化模型。抛弃式原

型解决对未来知识的局限性产生的不确定性，将未来置于现在进行推敲。

- i. 优点：
 - 1. 加强了和用户、客户的交流，提高满意度
 - 2. 适用于新颖的领域。
- ii. 缺点：
 - 1. 原型开发带来风险：如成本，耗尽项目时间资金
 - 2. 不舍得抛弃原型，使得低质量的代码进入最终产品导致产品低质量。
- iii. 使用范围：

大量不确定的新颖领域。如第一个智能手机...

螺旋模型



f) 螺旋模型：螺旋模型将软件开发活动组织为风险解决的迭代

- i. 优点：降低风险
- ii. 缺点：
 1. 风险解决使用原型，存在原型的风险
 2. 模型过于复杂，不利于管理。
- iii. 使用范围：

高风险大大规模软件系统开发。

g) **Rational 统一过程: RUP** 总结和借鉴传统上的各种有效经验, 建立最佳实践方法的集合, 并提供有效的过程定制手段, 允许开发者根据特定的需要定制一个有效的过程模型

i. 核心思想:

1. 迭代式开发, 这是过去被反复证明的最佳实践方法;
2. 管理需求, 重视需求工程中除了需求开发之外的需求管理活动;
3. 使用基于组件的体系结构, 它帮助建立一个可维护、易开发、易复用的软件体系结构;
4. 可视化建模, 利用 UML 进行建模;
5. 验证软件质量, 尽早和持续地开展验证, 以尽早发现缺陷, 降低风险和成本;

6. 控制软件变更, 适应 1990s 以后需求变更越来越重要的事实。

ii. 优点:

1. 吸收和借鉴了传统上的最佳实践方法, 尤其是其核心的 6 个实践方法, 能够保证软件开发过程的组织是基本有效和合理的。
2. RUP 依据其定制机制的不同, 可以适用于小型项目, 也可以适用于大型项目的开发, 适用面广泛。
3. RUP 有一套软件工程工具的支持, 这可以帮助 RUP 的有效实施。

iii. 缺点

1. 没有考虑交付之后的软件维护问题;
2. 裁剪和配置工作不是一个简单的任务, 无法保证每个项目都能定制一个有效的 RUP 过程

iv. 适用性: RUP 是重量级过程, 能够胜任大型软件团队开发大型项目时的活动组织。但 RUP 经过裁剪和定制, 也可以变为轻量级过程, 也能够胜任小团队的开发活动组织。

h) **敏捷过程:** 针对传统过程模型的缺陷和新的形势, 人们总结实践中的经验和最佳实践方法, 尝试建立轻量级过程方法。

i. 优点:

ii. 缺点:

iii. 适用性: 从敏捷联盟声明的思想和原则来看, 它们反映了 1990s 之后软件工程的发展趋势, 所以得到了广泛的应用, 尤其是能够适应于快速变化或者时间压力较大的项目。

对给定的场景, 判定使用的开发过程模型

软件工程知识体系的知识域

软件需求, 软件设计, 软件构造, 软件测试, 软件维护, 软件配置管理, 软件工程管理, 软件工程过程, 软件工程工具和方法, 软件质量

软件设计的七大原则

设计模式遵循的一般原则:

- 1.开-闭原则(Open-Closed Principle, OCP)
- 2.里氏代换原则(Liskov Substitution Principle,常缩写为 LSP)
- 3.依赖倒置原则(Dependence Inversion Principle)
- 4.接口隔离原则(Interface Segregation Principle, ISP)
- 5.合成/聚合复用原则(Composite/Aggregate Reuse Principle,CARP)

在一个新的对象里面使用一些已有的对象,使之成为新对象的一部分;新的对象通过这些向对象的委派达到复用已有功能的目的.这个设计原则有另一个简短的表述:要尽量使用合成/聚合,尽量不要使用继承.

- 6.迪米特法则(Law of Demeter LoD)
- 7.单一职责原则(Simple responsibility pinciple SRP)

二、Application 违反开闭原则，不对扩展开放。设计父类 class Application，添加一个 public 属性 os 和对应的 platform 就好

工厂模式：为对象的创建提供一个接口

三、违背迪米特原则 P233

四、都违背 Liskov 1.组合 2.

Kawata 同学希望测试 Akagi 同学设计的类

```
public class MyStack extends Vector {  
    public void push(Object element){  
        insertElementAt(element,0);  
    }  
    public Object pop(){  
        Object result = firstElement();  
        removeElementAt(0);  
        return result;  
    }  
}
```

- 1) 完成功能测试的测试用例的设计，说明思路
- 2) 给出相应的测试代码